CCSW 413 Software Design Patterns – Project (Stage 1 & Stage 2)

Team: Abdulaziz Almalki, Abdulelah Alzahrani, Reda Alamoudi	**Project:** Java Swing Calculator
Repository: https://github.com/joky711/Calculator-Design-Patterns	**Based on:** github.com/HouariZegai/Calculator

The original calculator keeps almost everything in one 585-line class, `CalculatorUI`. The two stages work on **different sections**: Stage 1 changes **how objects are created and loaded** (the theme loader and the button construction); Stage 2 changes **how the finished UI is put together and coloured** (converting theme data for Swing, and organising all widgets in a tree). The duplicated click-handler code is left on purpose for Stage 3 (Behavioral patterns).

Verification: the project compiles, and the original project test cases (`CalculatorUITest`: +, -, *, /, %, ^) all pass (6/6) on the Stage 2 code; the Dark theme and Scientific mode were checked on screen. The original project is by Houari Zegai (MIT licence); the licence file is kept in our repository.

Stage 1 (Creational) – Pattern 1: Singleton

Problem in the original code

`ThemeLoader` was a static helper class. Every call to `loadThemes()` created a new `ObjectMapper`, opened `application.yaml` and parsed it again, returning a new map each time. There was no single owner of the themes.

Solution

The constructor is now private and reads the file **once**. The only instance is kept in a `private static volatile` field and returned by `getInstance()`, using lazy initialization and double-checked locking. The UI calls `ThemeLoader.getInstance().getThemes()`.

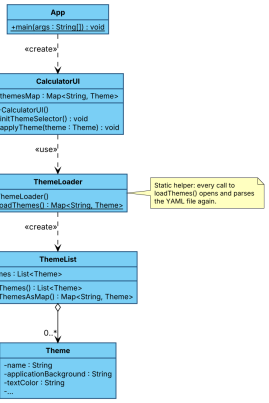


Figure 1: Singleton – class diagram before

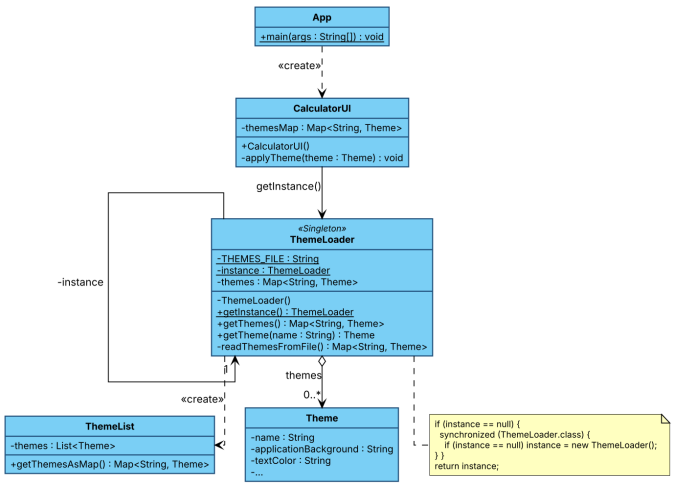


Figure 2: Singleton – class diagram after

Why this pattern and trade-offs

The themes come from one configuration file, so one shared, read-once source fits naturally, and any future class can reach it without passing the map around. **Trade-off:** a Singleton is global state and is harder to replace in unit tests; the alternative is creating one loader in `App.main` and passing it in (dependency injection).

Stage 1 (Creational) – Pattern 2: Factory Method

Problem in the original code

One method, `createButton(label, x, y)`, built all 23 keys the same way. The keys that are different were patched afterwards inside `initButtons()`: "=" was resized to two columns, "pow" got a smaller font, and the three scientific keys were hidden one by one.

Solution

Abstract creator `ButtonCreator` has `create(label, x, y)` with the common steps (bounds, cursor, focus), which calls the factory method `createButton(label)`. Concrete creators: `NumberButtonCreator`, `OperatorButtonCreator` (bold, tooltip), `EqualButtonCreator` (double width), `ScientificButtonCreator` (smaller font, hidden). `JButton` is the product.

```
public abstract class ButtonCreator {
    public JButton create(String label, int x, int y) {
        JButton btn = createButton(label);              // factory method
        btn.setBounds(x, y, getButtonWidth(), BUTTON_HEIGHT);
        ...
    }
    protected abstract JButton createButton(String label);
}
```

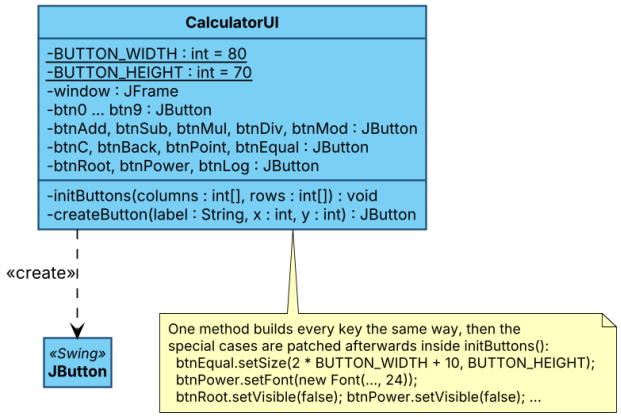


Figure 3: Factory Method – class diagram before

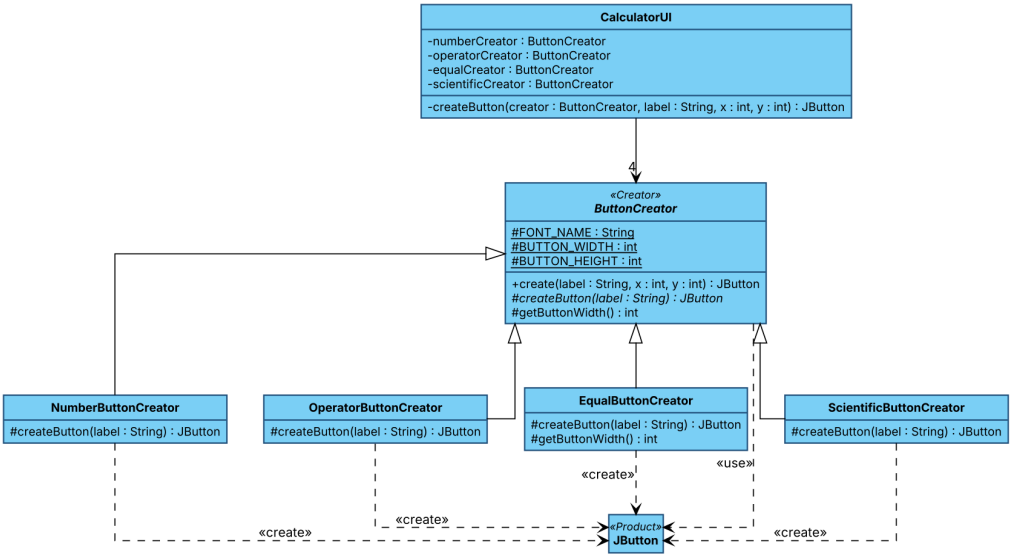


Figure 4: Factory Method – class diagram after

Why this pattern and trade-offs

The calculator has clear families of keys built in different ways; each family now has its own class and `CalculatorUI` only chooses the creator. A new kind of key (e.g. memory keys M+, M-) is a new subclass without editing existing code (Open/Closed). **Why not just pass width and font as parameters?** That works for the current keys, but every caller would need to know the settings of each key, and new behaviour (a hidden key, a tooltip, a different Swing class) would mean more parameters. A creator subclass keeps the rules of each family in one place. **Trade-off:** five small classes instead of one method.

Stage 2 (Structural) – Pattern 3: Adapter

Problem in the original code

`Theme` stores every colour as a hex string read from YAML (e.g. "0067c0"), but Swing needs `java.awt.Color`. `applyTheme()` converted the strings itself with 51 calls to `hex2Color()`, so the UI depended on the file format.

Solution

Target interface `ColorScheme` describes what the UI wants: ready `Color` objects. The object adapter `ThemeColorAdapter` implements it, wraps a `Theme` (adaptee) and converts each colour with `ColorUtil`. `Theme` itself is unchanged.

```
public class ThemeColorAdapter implements ColorScheme {
    private final Theme theme;                          // adaptee
    public ThemeColorAdapter(Theme theme) { this.theme = theme; }
    public Color getTextColor() { return hex2Color(theme.getTextColor()); }
    ...
}
```

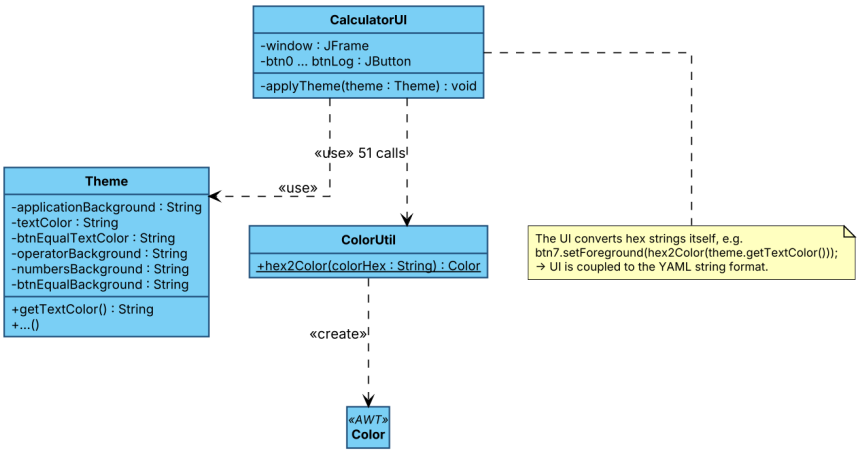


Figure 5: Adapter – class diagram before

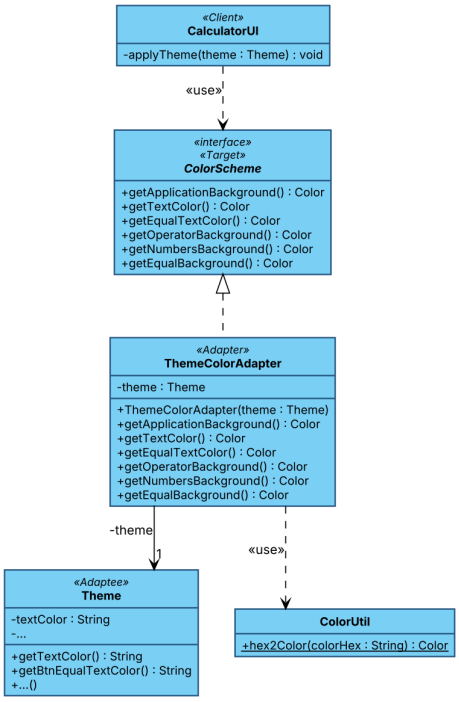


Figure 6: Adapter – class diagram after

Why this pattern and trade-offs

`Theme` is the data object that mirrors the YAML file (the parser fills it), so its format is set by the file, while Swing needs `Color`. The adapter keeps the UI independent of that file format, which is exactly the Adapter case. The UI now only knows `ColorScheme`, the conversion is written once per colour (6 methods instead of 51 calls), and a new theme source (JSON, database) only needs another adapter. **Trade-off:** one extra level of indirection; values are converted on each call (cheap, and could be cached).

Stage 2 (Structural) – Pattern 4: Composite

Problem in the original code

`CalculatorUI` kept 23 separate button fields plus the screen and two combo boxes. `applyTheme()` was 55 lines (one line per widget), and Scientific mode showed or hid the three scientific keys one by one. Adding a key meant editing several places.

Solution

Component interface `CalculatorComponent` (`applyColors`, `setVisible`); leaves `KeyComponent` (one button, coloured by its `KeyType`) and `FieldComponent` (screen or combo box); composite `ComponentGroup` forwards every call to its children. Tree: root → fields, numberKeys, operatorKeys, equalKeys, scientificKeys.

```
private void applyTheme(Theme theme) {
    ColorScheme colors = new ThemeColorAdapter(theme);   // Adapter
    window.getContentPane().setBackground(colors.getApplicationBackground());
    root.applyColors(colors);                            // Composite
}
```

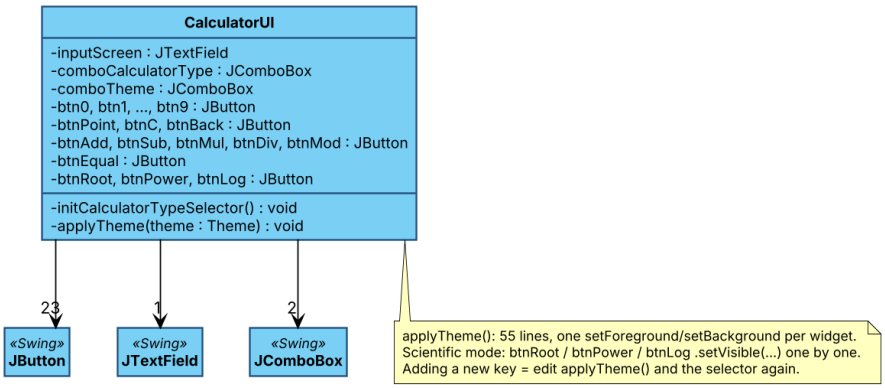


Figure 7: Composite – class diagram before

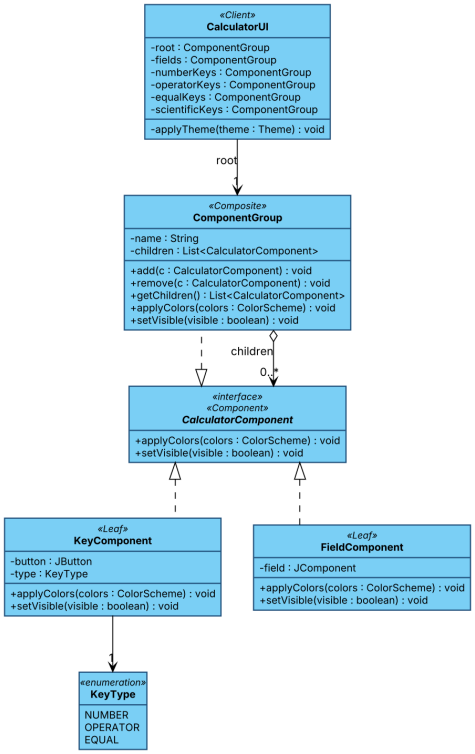


Figure 8: Composite – class diagram after

Why this pattern and trade-offs

The screen is a part–whole hierarchy, and one key and a whole group are treated the same way. `applyTheme()` went from 55 lines to 3, the 23 button fields were removed, and new keys are themed and shown/hidden automatically. **Trade-off:** a general interface hides leaf-specific features. `KeyComponent` still uses a small switch on `KeyType`; in Stage 3 this can be replaced (each creator returning its own component, or a Strategy for colouring).